

Submission Worksheet

Submission Data

Course: IT114-450-M2025

Assignment: IT114 Milestone 2 - RPS

Student: Alex P. (ap2869)

Status: Submitted | **Worksheet Progress:** 100%

Potential Grade: 10.00/10.00 (100.00%)

Received Grade: 0.00/10.00 (0.00%)

Started: 7/21/2025 1:48:24 PM

Updated: 7/21/2025 10:10:56 PM

Grading Link: <https://learn.ethereallab.app/assignment/v3/IT114-450-M2025/it114-milestone-2-rps/grading/ap2869>

View Link: <https://learn.ethereallab.app/assignment/v3/IT114-450-M2025/it114-milestone-2-rps/view/ap2869>

Instructions

1. Refer to Milestone2 of [Rock Paper Scissors](#)
 1. Complete the features
2. Ensure all code snippets include your ucid, date, and a brief description of what the code does
3. Switch to the Milestone2 branch
 1. `git checkout Milestone2`
 2. `git pull origin Milestone2`
4. Fill out the below worksheet as you test/demo with 3+ clients in the same session
5. Once finished, click "Submit and Export"
6. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
 1. `git add .`
 2. ``git commit -m "adding PDF"`
 3. `git push origin Milestone2`
 4. On Github merge the pull request from Milestone2 to main
7. Upload the same PDF to Canvas
8. Sync Local
 1. `git checkout main`
 2. `git pull origin main`

Section #1: (1 pt.) Payloads

Progress: 100%

≡ Task #1 (1 pt.) - Show Payload classes and subclasses

Progress: 100%

Details:

- Reqs from the document
 - Provided Payload for applicable items that only need client id, message, and type


```

// GameRoom
READY_READY, // trigger to tell the client to reset their whole local turn's ready status
// (saves network requests)
PHASE, // syncs current phase of execution (used as a switch to only allow certain logic
// to execute)
TURN, // example of taking a turn and syncing a turn action
SYNC_TURN, // quiet version of TURN, used to sync existing turn status of clients in a
// GameRoom
RESET_TURN, // trigger to tell client to reset their local turn status
// You, 17 minutes ago • 31 lines • 3 Patched projects
PATCH
ACTION
// #37-42 public enum payloadtype

```

snippet of payload types

 Saved: 7/21/2025 10:10:56 PM

≡ Part 2:

Progress: 100%

Details:

- Briefly explain the purpose of each payload shown in the screenshots and their properties

Your Response:

for e MESSAGE the purpose would be for communication. thr properties used are client Id, which shows he sender of the message message, which shows the content of the message and payloadtype, which sets the message.

for the rest its prerrt simple whaat they do .

 Saved: 7/21/2025 10:10:56 PM

Section #2: (4 pts.) Lifecycle Events

Progress: 100%

≡ Task #1 (0.80 pts.) - GameRoom Client Add/Remove

Progress: 100%

🖼 Part 1:

Progress: 100%

Details:

- Show the `onClientAdded()` code
- Show the `onClientRemoved()` code

```

public GameRoom(String name) {
    super(name);
}

/** {@inheritDoc} */
@Override
protected void onClientAdded(ServerThread sp) {
    // sync GameRoom state to new client
    syncCurrentPhase(sp);
    syncReadyStatus(sp);
    syncTurnStatus(sp);
}
<- #37-42 protected void onClientAdded(ServerThread sp)

```

snippet of clientAdded

```
/** @inheritDoc */
@Override
protected void onClientRemoved(ServerThread sp) {
    // Stops the timers so room can clean up
    LoggerUtil.INSTANCE.info("Player Removed, remaining: " + clientsInRoom.size());
    long removedClient = sp.getClientId();
    turnOrder.removeIf(player -> player.getClientId() == sp.getClientId());
    if (clientsInRoom.isEmpty()) {
        resetReadyTimer();
        resetTurnTimer();
        resetRoundTimer();
        onSessionEnd();
    } else if (removedClient == currentTurnClientId) {
        onTurnStart();
    }
}
} <- #46-59 protected void onClientRemoved(ServerThread sp)
```

snippet of clientRemoved

 Saved: 7/21/2025 5:30:33 PM

Part 2:

Progress: 100%

Details:

- Briefly note the actions that happen in `onClientAdded()` (app data should at least be synchronized to the joining user)
- Briefly note the actions that happen in `onClientRemoved()` (at least should handle logic for an empty session)

Your Response:

the method `onClientAdded` ensures that when a new client is added and automatically matched the current state of the game. `syncCurrentPhase`, syncs the game phase `syncReadyStatus`, syncs the readiness of the player `syncTurnStatus`, syncs if its the players turn or not.

the method `onClientRemoved` logs the changes made and removes the client from when its their turn or not. this method also stops the game if all players are removed and ends session. if the player is removed and there are still remaining players the method ensures that the person who is next goes. this would make the game run smoothly.

 Saved: 7/21/2025 5:30:33 PM

Task #2 (0.80 pts.) - GameRoom Session Start

Progress: 100%

Details:

- Reqs from document
 - First round is triggered
- Reset/set initial state

Part 1:

Progress: 100%

Details:

- Show the snippet of `onSessionStart()`

```
89      /** {@inheritDoc} */
90      @Override
91      protected void onSessionStart() {
92          LoggerUtil.INSTANCE.info(message:"onSessionStart() start");
93          changePhase(Phase.IN_PROGRESS);
94          currentTurnClientId = Constants.DEFAULT_CLIENT_ID;
95          setTurnOrder();
96          round = 0;
97          LoggerUtil.INSTANCE.info(message:"onSessionStart() end");
98          onRoundStart();
99      } <- #91-99 protected void onSessionStart()
100
```

snippet of Session start

 Saved: 7/21/2025 5:44:04 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the logic that occurs here (i.e., setting up initial session state for your project) and next lifecycle trigger

Your Response:

the log starts the game that is programmed . in this case its RPS IN_PROGRESS is then set, which shows that the game is active. currentTurnClientId then resets to a default value because it isn't anyone's turn yet. the method setTurnOrder is then called and sets the players on their turn. the RoundStart is then called and the first round of the game begins

 Saved: 7/21/2025 5:44:04 PM

Task #3 (0.80 pts.) - GameRoom Round Start

Progress: 100%

Details:

- Reqs from Document
 - Initialize remaining Players' choices to null (not set)
 - Set Phase to "choosing"
 - GameRoom round timer begins

Part 1:

Progress: 100%

Details:

- Show the snippet of `onRoundStart()`

```
/** @inheritDoc */  
@Override  
protected void onRoundStart() {  
    LoggerUtil.INSTANCE.info(message:"onRoundStart() start");  
    resetRoundTimer();  
    resetTurnStatus();  
    round++;  
    relay(sender:null, String.format(format:"Round %d has started", round));  
    // startRoundTimer(); Round timers aren't needed for turns  
    // if you do decide to use it, ensure it's reasonable and based on the number of  
    // players  
    LoggerUtil.INSTANCE.info(message:"onRoundStart() end");  
    onTurnStart();  
}
```

Snippet of RoundStart

 Saved: 7/21/2025 6:41:53 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the logic that occurs here (i.e., setting up the round for your project)

Your Response:

the code first starts the round (1st round out of 3) a round timer is already set and resetRoundTimer restarts the timers. when the round begins ResetTurnStatus ensures that all players begin with a clean start. then a message is sent to all players that the round has started. finally it logs the end of the round set up it then calls onTurnStart methods to begin.

 Saved: 7/21/2025 6:41:53 PM

Task #4 (0.80 pts.) - GameRoom Round End

Progress: 100%

Details:

- Reqs from Document
 - **Condition 1:** Round ends when round timer expires
 - **Condition 2:** Round ends when all active Players have made a choice
 - All Players who are not eliminated and haven't made a choice will be marked as eliminated
 - Process Battles:

- ☐ Round-robin battles of eligible Players (i.e., Player 1 vs Player 2 vs Player 3 vs Player 1)
 - ☐ Determine if a Player loses if they lose the "attack" or if they lose the "defend" (since each Player has two battles each round)
 - ☐ Give a point to the winning Player
 - ☐ Points will be stored on the Player/User object
 - ☐ Sync the points value of the Player to all Clients
 - ☐ Relay a message stating the Players that competed, their choices, and the result of the battle
 - ☐ Losers get marked as eliminated (Eliminated Players stay as spectators but are skipped for choices and for win checks)
- ☐ Count the number of non-eliminated Players
 - ☐ If one, this is your winner (onSessionEnd())
 - ☐ If zero, it was a tie (onSessionEnd())
 - ☐ If more than one, do another round (onRoundStart())

Part 1:

Progress: 100%

Details:

- Show the snippet of `onRoundEnd()`

```
/** @inheritDoc */
@Override
protected void onRoundEnd() {
    LoggerUtil.INSTANCE.info(message:"onRoundEnd() start");
    resetRoundTimer(); // reset timer if round ended without the time expiring

    LoggerUtil.INSTANCE.info(message:"onRoundEnd() end");
    if (round >= 3) {
        onSessionEnd();
    } else {
        You, 3 days ago • adding Round Turns Files for game
        onRoundStart();
    }
} <- #158-168 protected void onRoundEnd()
```

snippet of RoundEnd

 Saved: 7/21/2025 7:43:00 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the logic that occurs here (i.e., cleanup, end checks, and next lifecycle events)

Your Response:

the code first starts the round ending process. resetRoundTimer resets the timer that is set in place once the round start onSessionEnd calls the end of the game round and onRoundStart will

begin a new round. after 3 rounds the round will be over.



Saved: 7/21/2025 7:43:00 PM

≡ Task #5 (0.80 pts.) - GameRoom Session End

Progress: 100%

Details:

- Reqs from Document
 - **Condition 1:** Session ends when one Player remains (they win)
 - **Condition 2:** Session ends when no Players remain (this is a tie)
 - Send the final scoreboard to all clients sorted by highest points to lowest (include a game over message)
 - Reset the player data for each client server-side and client-side (do not disconnect them or move them to the lobby)
 - A new ready check will be required to start a new session

📄 Part 1:

Progress: 100%

Details:

- Show the snippet of `onSessionEnd()`

```
/** @inheritDoc */
@Override
protected void onSessionEnd() {
    LoggerUtil.INSTANCE.info(message:"onSessionEnd() start");
    turnOrder.clear();
    currentTurnClientId = Constants.DEFAULT_CLIENT_ID;
    resetReadyStatus();
    resetTurnStatus();
    changePhase(Phase.READY);
    LoggerUtil.INSTANCE.info(message:"onSessionEnd() end");
}
// end lifecycle methods
// send/sync data to server/thread(c)
private void sendResetTurnStatus() {
    clientsInRoom.values().forEach(spInRoom -> {
        boolean failedToSend = !spInRoom.sendResetTurnStatus();
        if (failedToSend) {
            removeClient(spInRoom);
        }
    });
}
// #184-189 clientsInRoom.values().forEach
// #183-190 private void sendResetTurnStatus()
```

snippet of SessionEnd



Saved: 7/21/2025 7:47:54 PM

≡ Part 2:

Progress: 100%

Details:

- Briefly explain the logic that occurs here (i.e., cleanup/reset, next lifecycle events)

Your Response:

turnOver.clear clears the turnOver list to remove players order. then currentTurnClientID is set to its default value, which means no turn yet. restReadyTurn and restTurn status resets the players ready and turn status. the game phase is then set back to READY, ready for a new round/session.



Saved: 7/21/2025 7:47:54 PM

Section #3: (4 pts.) Gameroom User Action And State

Progress: 100%

≡ Task #1 (2 pts.) - Choice Logic

Progress: 100%

Details:

- Reqs from document
 - Command: `/pick <[r,p,s]>` (user picks one)
 - ☐ GameRoom will check if it's a valid option
 - ☐ GameRoom will record the choice for the respective Player
 - ☐ A message will be relayed saying that "X picked their choice"
 - ☐ If all Players have a choice the round ends

📁 Part 1:

Progress: 100%

Details:

- Show the code snippets of the following, and clearly caption each screenshot
- Show the Client processing of this command (process client command)
- Show the ServerThread processing of this command (process method)
- Show the GameRoom handling of this command (handle method)
- Show the sending/syncing of the results of this command to users (send/sync method)
- Show the ServerThread receiving this data (send method)
- Show the Client receiving this data (process method)

```
//  
* Listens for keyboard input from the user  
*/  
private void listenToInput() {  
    try (Scanner si = new Scanner(System.in)) {  
        LoggerUtil.INSTANCE.info(message:"Waiting for input"); // moved here to avoid console spam  
        while (isRunning) { // Run until isRunning is false  
            String userInput = si.nextLine();  
            if (!processClientCommand(userInput)) {  
                sendMessage(userInput);  
            }  
        }  
    } catch (IOException ioException) {  
        LoggerUtil.INSTANCE.severe(message:"Error in listenToInput()", ioException);  
        // ioException.printStackTrace();  
    }  
    LoggerUtil.INSTANCE.info(message:"listenToInput thread stopped");  
}
```

snippet of client process command

snippet of game setup commands and round 1, also timer



Saved: 7/21/2025 10:04:26 PM

Part 2:

Progress: 100%

Details:

Include the link to the Pull Request (should end in `/pull/#`)

URL #1

<https://github.com/ap2869/IT-114-450/pull/12>



URL

<https://github.com/ap2869/IT-114>



Saved: 7/21/2025 10:04:26 PM

Task #2 (0.33 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the [WakaTime.com](https://wakatime.com) Dashboard
- Click **Projects** and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary

Files		Branches	
4 hrs 51 mins	Project/server/MainActivity.java	5 hrs 5 mins	Milestone2-Project
1 hr 9 mins	Project/Client/Client.java	1 hr 5 mins	Milestone2-ready-check
59 mins	Project/Server/ServerThread.java	1 hr 1 min	Milestone2-bonus-PhasedMaze
56 mins	Project/Common/Payload.java	1 hr	main
53 mins	PROJECT11/Server/Server.java	11 mins	Milestone2-Example-Project
52 mins	PROJECT11/Client/Client.java	5 mins	Milestone2-Second-Turns
52 mins	Project/Common/Thread.java		PhasedDance
50 mins	Project/Server/MainActivity.java		
40 mins	Project/Common/PayloadType.java		
6 mins	Project/Common/TextK.java		
8 mins	Server.java		
4 mins	Project/Common/Command.java		
4 mins	Project/Server/MainActivityThread.java		
3 mins	Project/Server/Server.java		
2 mins	Project/server/Room.java		
1 min	Project/Server/MainActivityThread.java		
1 min	PROJECT11/Server/ServerThread.java		
1 min	PROJECT11/Client/Client.java		
1 min	Project/Common/Thread.java		
1 min	Project/Common/ReadyPayload.java		
1 min	Project/Server/MainActivityThread.java		

snippet of waka files

Projects • IT-114-450

total 35 hrs 41 mins

9 hrs 6 mins over the Last 7 Days in IT-114-450 under all branches.



Saved: 7/21/2025 10:04:59 PM

≡ Task #3 (0.33 pts.) - Reflection

Progress: 100%

⇒ Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

while doing this milestone i have learned alot. from the previous sections wwe had the nessary set of of socckets and with that learning how it is implemented with networking programs was fairly interesting. it also game me better unserstanding on how to handle multiclients and shared gamestats. with the help of the modules i was able to learn how to structure turn based logic, user/player input, and design clean ojbet oriented code.



Saved: 7/21/2025 2:21:45 PM

⇒ Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

the easiest part of the assignment was messing up one small thing. either somethignwas out of place or i just forgot sto fix a spelling error. case sensetivity was also easy to forget. but all in all i disd have alot of fun doing this project and if we had much mor time not just a week i probally would have choosen something a little more challanging but i had to start with probally the less time consuming one. kinda.



Saved: 7/21/2025 1:51:59 PM

Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

there were alot of difficulty from doing this project. but the most difficult part was actually the testing of the project. everytime i had to make a change or implemented a line of code it was a repition of `./build.sh -> ./run.sh` and so on and so forth. just the set up took the majority of the time and it was done for every little change . i ahd to implement the fix and test it everytime which at times got boring and just repetative nut when it finally worked i was releaved.



Saved: 7/21/2025 1:49:47 PM